

Reset timelines

Reset puts a flip-flop into a known state, usually zero when rst_n is low

Mid-cycle, edge, release

- Starter scenario: drive D to one, take a posedge so both q_sync and q_async become one
- Assert rst_n low mid-cycle, the async flop clears immediately
- At a posedge with rst_n still low, both clear
- Release rst_n high, then capture new data on a later edge
- A short mid-cycle pulse may be ignored by sync reset if it ends before the sampling edge

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Reset timelines

INTERACTIVE TOOL

Reset strategy timelines

Compare **synchronous** vs **asynchronous** reset on the same stimulus: assert `rst_n` between clock edges and watch when `q` clears. Starter: both FFs hold `q=1`, then an async-style mid-cycle reset pulse.

Starter example: capture `q=1`, then assert `rst_n=0` mid-cycle — async clears immediately; sync still shows 1 until the next posedge.

Load starter example

Challenges 0/22

Pick one

Quiz: sync reset

A synchronous reset is typically...

☐ sampled only on the clock edge (inside always @(posedge clk))

☐ sensitive to `rst_n` with no clock

☐ the same as `$finish`

☐ analog only

Start

Show hint

Check

Next

Stop

Idle

Reset timelines starter

Workbook practice

- In the workbook track, sketch `clk`, `rst_n`
- Write the `always` block for each style
- For `rst_n` low at posedge, do both FFs end at zero?
- Name one pitfall: releasing async reset too close to a clock edge without recovery time

Pitfalls to watch

- Do not assume sync and async behave the same off the edge
- Rst_n naming implies active low, not “never reset.” And remember
- Real SoCs still need reset trees, CDC

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, finish a few challenges after the starter
- On paper, draw one mid-cycle reset wave
- When you are ready, take the short quiz, then continue to clock enable versus gated clock